

P1102R2

Down with ()!

Published Proposal, 2020-12-11

This version:

<http://wg21.link/P1102R2>

Authors:

[Alex Christensen](#) (Apple)

[JF Bastien](#) (Toyota Research Institute—Advanced Development)

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

<https://github.com/achristensen07/papers/blob/master/source/P1102r2.bs>

Abstract

A proposal for removing unnecessary `()`'s from C++ lambdas.

Table of Contents

- 1 Revision History
- 2 Introduction and motivation
- 3 Impact
- 4 Wording

References

Informative References

§ 1. Revision History

EWG discussed this change as [\[EWG135\]](#) in [Lenexa](#) and voted 15 to 1 on forwarding to core. It became [\[CWG2121\]](#), discussed in [Kona](#) and needed someone to volunteer wording. This paper presents text that implements that decision, for consideration.

[\[P1102R0\]](#) was published in June 2018. It was discussed on the EWG reflector in June 2018, Nina Ranns provided feedback, and EWG chair agreed that the paper should move to CWG directly given previous polls.

[\[P1102R1\]](#) responded to feedback about ambiguous requires-clauses from Hubert Tong and was discussed in a CWG teleconference on 2020-12-07.

P1102R2 incorporates the feedback given by CWG during that teleconference.

§ 2. Introduction and motivation

Currently, C++ lambdas with no parameters do not require a parameter declaration clause. The specification even contains this language in [\[expr.prim.lambda\]](#) section 8.4.5 ¶4:

If a lambda-expression does not include a lambda-declarator, it is as if the lambda-declarator were `()`.

This allows us to omit the unused `()` in simple lambdas such as this:

```
std::string s1 = "abc";
auto withParen = [s1 = std::move(s1)] () {
    std::cout << s1 << '\n';
};

std::string s2 = "abc";
auto noSean = [s2 = std::move(s2)] { // Note no syntax error.
    std::cout << s2 << '\n';
};
```

These particular lambdas have ownership of the strings, so they ought to be able to mutate it, but `s1` and `s2` are const (because the `const` operator is declared `const` by default) so we need to add the `mutable` keyword:

```

std::string s1 = "abc";
auto withParen = [s1 = std::move(s1)] () mutable {
    s1 += "d";
    std::cout << s1 << '\n';
};

std::string s2 = "abc";
auto noSean = [s2 = std::move(s2)] mutable { // Currently a syntax error.
    s2 += "d";
    std::cout << s2 << '\n';
};

```

Confusingly, the current Standard requires the empty parens when using the `mutable` keyword. This rule is unintuitive, causes common syntax errors, and clutters our code. When compiling with clang, we even get a syntax error that indicates the compiler knows exactly what is going on:

```

example.cpp:11:54: error: lambda requires '()' before 'mutable'
auto noSean = [s2 = std::move(s2)] mutable { // Currently a syntax error.
                                ^
                                ()
1 error generated.

```

This proposal would make these parentheses unnecessary like they were before we added `mutable`. This will apply to:

- lambda template parameters
- `constexpr`
- `mutable`
- `constexpr`
- Exception specifications and `noexcept`
- attributes
- trailing return types
- `requires`

§ 3. Impact

This change will not break existing code.

§ 4. Wording

Modify Lambda expressions [**expr.prim.lambda**] as follows:

```
lambda-expression :  
    lambda-introducer lambda-declaratoropt compound-statement  
    lambda-introducer < template-parameter-list > requires-clauseopt  
        lambda-declaratoropt compound-statement  
lambda-introducer :  
    [ lambda-captureopt ]  
lambda-declarator:  
    lambda-specifiers  
    ( parameter-declaration-clause ) decl-specifier-seqopt lambda-specifiers  
        noexcept-specifieropt attribute-specifier-seqopt trailing-return-typeopt requires-  
clauseopt  
    lambda-specifiers:  
        decl-specifier-seqopt noexcept-specifieropt attribute-specifier-seqopt trailing-return-  
typeopt
```

The _{opt} after *lambda-declarator* is removed because *lambda-declarator* can now be empty and the _{opt} would be redundant. *lambda-specifiers* is given a name to reduce redundancy in the grammar.

Modify §4 to clarify that all *lambda-expressions* now have a *lambda-declarator* but it may be empty:

If a *lambda-expression* ~~does not include a~~ includes an empty *lambda-declarator*, it is as if the *lambda-declarator* were `()`. [...]

§ References

§ Informative References

[CWG2121]

EWG. [More flexible lambda syntax](https://wg21.link/cwg2121). 6 May 2015. drafting. URL: <https://wg21.link/cwg2121>

[EWG135]

Herb Sutter. [\[tiny\] Mutable is part of a lambda-declarator, so when a lambda is mutable, the parentheses aren't optional](https://wg21.link/ewg135). New. URL: <https://wg21.link/ewg135>

[P1102R0]

Alex Christensen, JF Bastien. [Down with \(\)!](https://wg21.link/p1102r0). 20 June 2018. URL: <https://wg21.link/p1102r0>

[P1102R1]

Alex Christensen, JF Bastien. [Down with \(\)!](https://wg21.link/p1102r1). URL: <https://wg21.link/p1102r1>